

AutoSAGE: Input-Aware CUDA Scheduling for Sparse GNN Aggregation and CSR Attention

Aleksandar Stanković

Faculty of Technical Sciences, University of Novi Sad

Novi Sad, Serbia

Email: stankovic.sv25.2022@uns.ac.rs

Artifacts and code: <https://github.com/SV25-22/AutoSAGE>

Abstract—Graph neural networks (GNNs) rely on sparse matrix–matrix operations such as sparse–dense SpMM and sampled dense–dense SDDMM over CSR, whose performance can vary widely with graph structure, feature width and GPU architecture. A single static CUDA kernel therefore under-utilizes some workloads and devices.

We propose *AutoSAGE*, an input-aware CUDA scheduler for CSR-based GNN aggregation. AutoSAGE chooses at runtime among a small set of SpMM/SDDMM kernel variants using a cheap analytical estimate followed by short on-device micro-probes, and protects against regressions with a simple guardrail and a persistent replay cache. It supports both SpMM and SDDMM and composes into a CSR attention pipeline (SDDMM $\rightarrow$ row-softmax $\rightarrow$ SpMM).

On REDDIT and OGBN-PRODUCTS, AutoSAGE matches cuSPARSE-based baselines in bandwidth-bound regimes and delivers consistent speedups at small feature widths. On synthetic graphs that stress sparsity and degree skew, it achieves up to $4.7\times$ kernel-level speedups. We release CUDA sources, PyTorch bindings and a reproducible evaluation harness.

Index Terms—graph neural networks, sparse matrix multiplication, CUDA, GPU scheduling, SpMM, SDDMM, attention

I. INTRODUCTION

Graph neural networks (GNNs) have become a standard tool for learning on relational data, including communication networks, social and recommendation graphs, and power-system topologies. In many implementations, the dominant cost during training is the sparse aggregation stage, typically realized as sparse–dense matrix multiplication (SpMM) or sampled dense–dense multiplication (SDDMM) over a compressed sparse row (CSR) representation.

The performance of these kernels is highly sensitive to both the input graph and the underlying GPU. Heavy-tailed degree distributions, varying feature widths and resource constraints such as registers and shared memory can drastically change which tiling and mapping strategy is optimal. In practice, however, many systems rely on a single static kernel or vendor implementation across all graphs and devices, leading to significant inefficiencies on a subset of workloads.

Fully general autotuning tools are often too expensive or complex to integrate into production GNN systems. Practitioners instead want a solution that is input-aware, safe (no regressions relative to a strong baseline), low-overhead and easy to deploy.

A. Our Approach

In this work we address this gap with *AutoSAGE*, an input-aware scheduler for sparse GNN aggregation on GPUs. AutoSAGE focuses on a narrow but important class of operations: CSR SpMM, SDDMM and the corresponding CSR attention pipeline, and augments them with a small number of hand-written CUDA kernel variants. At runtime, AutoSAGE:

- 1) extracts a compact set of features from the input graph and operation,
- 2) uses a cheap analytical estimate to shortlist promising kernel variants,
- 3) times the most promising candidates on a small induced subgraph using micro-benchmarks, and
- 4) applies a guardrail to ensure that the final choice does not regress relative to a baseline kernel.

To avoid repeating this process, AutoSAGE stores decisions in a persistent cache keyed by device, graph signature, feature width and operation, enabling deterministic replay in subsequent runs.

B. Contributions

The main contributions of this paper are:

- **Design of AutoSAGE.** An input-aware scheduler for CSR SpMM and SDDMM that combines analytical estimates, micro-probes and a guardrail mechanism to select kernels at runtime while avoiding performance regressions.
- **Kernel variants for SpMM, SDDMM and CSR attention.** A small set of CUDA kernels that cover common regimes, including warp-per-row and CTA-per-hub mappings and an optional `vec4` vectorized path, composed into a CSR attention pipeline.
- **Empirical evaluation.** An evaluation on two widely used GNN benchmarks and on synthetic graphs that stress sparsity and degree skew, showing up to $4.700\times$ speedups in challenging sparse scenarios while matching vendor baselines elsewhere.
- **Practical implementation.** A PyTorch integration with simple environment-variable toggles, logging and replay, and a reproducible harness and cache logs to support further experimentation.

II. BACKGROUND

A. GNN Aggregation as SpMM and SDDMM

Many message-passing GNNs can be expressed in terms of linear-algebra operations on the adjacency matrix A and node feature matrices. We use a CSR representation (`rowptr`, `colind`, `val`) for A .

SpMM computes $C = AB$, where $A \in \mathbb{R}^{N \times N}$ is sparse and $B \in \mathbb{R}^{N \times F}$ is dense. Each row of C is an aggregation of the features of neighboring nodes. *SDDMM* computes edge-wise scores restricted to the sparsity pattern of A , e.g.

$$\tilde{A}_{ij} = \langle X_i, Y_j \rangle \quad \text{for } (i, j) \in \mathcal{S}(A),$$

and is used for attention mechanisms and edge feature computations. A common CSR attention pipeline applies SDDMM followed by a row-wise softmax and a final SpMM.

B. Related Work

Sparse GNN kernels. Several works optimize the underlying sparse kernels used in GNNs. GE-SpMM [1] and FusedMM [2] improve CSR SpMM and SDDMM using coalesced caching, vectorization and load balancing, while FeatGraph [3] maps GNN operators to TVM templates. Recent work on GPU sparse kernels includes Sputnik [14], GraphBLAST [15], DTC-SpMM [16], Voltrix-SpMM [17] and HR-SpMM [18].

GNN systems and acceleration. At the system level, GNNAdvisor [4] and AdaptGear [5] adapt execution at the runtime or subgraph level. TC-GNN [22] targets Tensor Cores, while Graphiler [23] compiles message-passing to a data-flow IR. Systems such as NextDoor [19], BGL [20] and PipeGCN [21] optimize data loading, sampling and communication. Surveys and evaluations [24], [25] highlight the growing diversity of GNN systems. In contrast to these end-to-end frameworks, AutoSAGE focuses on the kernel level: it selects among a small set of SpMM/SDDMM implementations while treating the surrounding GNN as a black box.

Autotuning and code generation. AutoTVM and Ansor [9], [10] search large schedule spaces with learned cost models. GraphIt and its GPU backend G2 [12], [13] expose scheduling dimensions for graph algorithms and support autotuning at the algorithm level, while Triton [11] offers templates for dense kernels. AutoSAGE takes a complementary approach: instead of searching over a very large schedule space, it chooses between a handful of pre-defined sparse kernels using low-cost runtime information and a guardrail.

III. DESIGN OF AUTOSAGE

A. Goals and Overview

AutoSAGE aims to (i) adapt to each combination of graph, feature width and operation (SpMM or SDDMM), (ii) avoid performance regressions relative to a strong baseline, and (iii) keep runtime decision overhead small compared to a full iteration.

To this end, AutoSAGE provides a small set of kernel templates for SpMM and SDDMM and a runtime scheduler that follows an *estimate* $\rightarrow$ *micro-probe* $\rightarrow$ *guardrail* pattern.

B. Kernel Templates

We implement two main families of kernels for CSR SpMM and an analogous set for SDDMM:

- **Warp-per-row SpMM.** A baseline kernel assigns one warp to each row and tiles the feature dimension into chunks $f_{\text{tile}} \in \{32, 64, 128, \dots\}$, which works well when degrees are relatively balanced and the feature width is moderate or large.
- **CTA-per-hub SpMM.** To handle graphs with a small number of very high-degree nodes, a hybrid kernel assigns a full CTA to “hub” rows above a configurable degree threshold and keeps warp-per-row mapping for the remaining rows.
- **SDDMM and CSR attention.** For SDDMM we use row-wise dot products with the same tiling and mapping options, together with a numerically stable CSR row-softmax to build a full CSR attention pipeline.

All kernels have scalar and optional `vec4` vectorized variants. The `vec4` path requires that the feature width F be divisible by four and that the data be 16-byte aligned.

TABLE I: Kernel variants and their typical regimes. F denotes feature width.

Variant	Row mapping	Typical regime
SpMM: warp-per-row	1 warp / row	Balanced degrees, mid/large F
SpMM: CTA-per-hub	1 CTA / heavy row	Hub-skew, small/mid F
SDDMM: rowwise dot	1 warp / row	Varies with F

C. Runtime Scheduler

The runtime scheduler chooses a kernel variant for each call. Given a CSR matrix A and feature width F , AutoSAGE first extracts simple features such as the number of rows and non-zeros, degree statistics, operation type (SpMM or SDDMM) and basic device capabilities. A lightweight roofline-style estimate uses these features to discard kernel variants that are unlikely to perform well, leaving a small set of candidates.

For the remaining candidates, AutoSAGE runs short micro-benchmarks on an induced subgraph consisting of a small fraction of rows (by default 2–3%, with a minimum of 512 rows). Each candidate is timed for a fixed number of iterations under a wall-time cap, yielding a measured latency t for each variant. We denote by t_b the latency of a chosen baseline kernel and by t^* the best observed candidate time.

The guardrail compares the best candidate to the baseline and accepts a candidate only if

$$t^* \leq \alpha t_b,$$

where $\alpha \leq 1$ is a user-configurable threshold (default $\alpha = 0.95$). Otherwise AutoSAGE falls back to the baseline. With $\alpha \leq 1$, this rule ensures a simple non-regression guarantee: the chosen kernel is never slower than the baseline under identical inputs and device.

D. Caching and Replay

To avoid repeated probing, AutoSAGE stores the chosen kernel in a cache keyed by (device signature, graph signature, feature width, operation). On subsequent runs with the same key, the scheduler directly reuses the cached decision. A replay mode allows users to disable probing and rely only on existing cache entries, which is useful for production deployments.

IV. IMPLEMENTATION

We implement AutoSAGE as a CUDA/C++ library with PyTorch bindings. The main components are:

- **Kernel sources and launcher.** Separate CUDA files handle light rows, hubs and SDDMM, with a shared launcher that configures block sizes and tiling.
- **PyTorch interface.** Functions such as `autosage::spmm_csr`, `autosage::spmm_csr_split` and `autosage::sddmm_csr` are exposed via the `torch.ops.autosage` namespace.
- **Configuration and logging.** Environment variables control probe fractions, time caps, tiling and split thresholds; each run produces CSV and JSON logs that record kernel choices, timings and device metadata to support reproducibility.

V. EXPERIMENTAL METHODOLOGY

A. Hardware and Software

All experiments are conducted on a single node with $8 \times$ NVIDIA A800-SXM4-40GB GPUs (compute capability 8.0, 108 SMs per GPU, 40 GB HBM each). The host machine has $2 \times$ Intel Xeon Platinum 8378A CPUs (2 sockets with 32 cores each, 128 hardware threads in total) and 1.0 TiB of RAM.

The software stack consists of Ubuntu 22.04.5 LTS (Linux 5.15), Python 3.12.2, PyTorch 2.8.0+cu128, cuDNN 9.1.2 and the CUDA 12.4 toolkit (nvcc V12.4.131). Multiple CUDA toolkits are installed on the node; all builds in this paper use nvcc 12.4.

B. Datasets

We use two widely used GNN benchmarks:

- **REDDIT** [6], a social-network graph with community structure and moderate size.
- **OGBN-PRODUCTS** [8], a large product co-purchasing graph from the Open Graph Benchmark.

To better understand behavior under controlled sparsity patterns, we also generate synthetic graphs:

- **Erdős-Rényi graphs** with $N = 200,000$ and edge probability $p = 2 \times 10^{-5}$;
- **Hub-skewed graphs** with $N = 200,000$ and a small set of very high-degree nodes.

C. Baselines and Metrics

The main baseline for SpMM is the cuSPARSE implementation accessed through PyTorch; for SDDMM we use a gather-dot implementation. We report median kernel latencies over 10–15 iterations after a warm-up, and focus on speedup relative to the baseline. Unless stated otherwise, the guardrail is set to $\alpha = 0.95$. We separately discuss the cost of probing and the effect of varying α .

VI. RESULTS

A. Overall Behavior

Across real and synthetic graphs, AutoSAGE exhibits two typical regimes: on real graphs and large feature widths, SpMM is memory-bandwidth bound and AutoSAGE usually selects the baseline kernel, yielding near-parity performance; at smaller feature widths and on synthetic graphs that emphasize sparsity or degree skew, AutoSAGE frequently selects specialized kernels and achieves substantial speedups.

B. Real Graphs

Tables I and II summarize the results on REDDIT and OGBN-PRODUCTS with guardrail $\alpha = 0.95$.

TABLE II: REDDIT (PyG) with guardrail $\alpha = 0.95$.

F	choice	baseline (ms)	chosen (ms)	speedup
64	autosage	16.241	15.093	1.076
128	baseline	38.115	38.153	0.999
256	baseline	89.801	89.778	1.000

a) REDDIT: At $F = 64$, AutoSAGE selects a non-baseline kernel and provides a modest improvement. At larger feature widths the operation becomes bandwidth-bound and the scheduler correctly defers to the baseline.

TABLE III: OGBN-PRODUCTS with guardrail $\alpha = 0.95$.

F	choice	baseline (ms)	chosen (ms)	speedup
64	autosage	27.534	26.385	1.044
128	baseline	56.152	56.161	1.000
256	baseline	114.550	114.385	1.001

b) OGBN-PRODUCTS: We observe a similar pattern on OGBN-PRODUCTS: AutoSAGE finds gains at small F and matches the baseline at larger F .

C. Synthetic Stressors

To better expose the effect of sparsity and degree skew, we evaluate AutoSAGE on synthetic graphs.

a) *Erdős-Rényi graphs*: Table III shows results on an Erdős-Rényi graph with $N = 200,000$ and $p = 2 \times 10^{-5}$.

TABLE IV: Erdős-Rényi synthetic graph ($N=200,000$, $p=2 \times 10^{-5}$).

F	choice	baseline (ms)	chosen (ms)	speedup
64	autosage	1.523	0.323	4.720
128	autosage	1.720	0.563	3.055
256	autosage	2.231	1.067	2.091

In this regime, AutoSAGE consistently chooses specialized kernels and achieves speedups of up to $4.700\times$ over the baseline.

b) *Hub-skewed graphs*: Table IV reports results on a hub-skewed graph with $N=200,000$, $k=4$ and $h=0.15$.

TABLE V: Hub-skew synthetic graph.

F	choice	baseline (ms)	chosen (ms)	speedup
64	autosage	10.190	8.976	1.135
128	autosage	19.267	18.074	1.066
256	baseline	37.362	37.351	1.000

Here the CTA-per-hub variant is particularly effective at medium feature widths, whereas large feature widths again become bandwidth-bound.

D. Sensitivity to Guardrail and Feature Width

We next study the effect of the guardrail parameter and feature width on REDDIT and OGBN-PRODUCTS.

a) *Guardrail parameter on REDDIT*: Table V shows REDDIT results with a stricter guardrail $\alpha = 0.98$.

TABLE VI: REDDIT with guardrail $\alpha = 0.98$.

F	choice	baseline (ms)	chosen (ms)	speedup
64	autosage	16.222	15.031	1.079
128	autosage	38.179	36.951	1.033
256	baseline	89.706	89.735	1.000

With $\alpha = 0.98$, AutoSAGE becomes more conservative but still captures consistent gains at $F = 64$ and $F = 128$, while falling back to the baseline at large F .

b) *Feature-width sweeps*: Tables VI and VII summarize wider sweeps over F for REDDIT and OGBN-PRODUCTS.

TABLE VII: REDDIT: feature-width sweep.

F	choice	baseline (ms)	chosen (ms)	speedup
32	autosage	7.871	6.614	1.190
64	autosage	16.257	15.070	1.079
96	baseline	26.486	26.584	0.996
128	baseline	38.482	38.074	1.011
192	baseline	63.256	63.328	0.999
256	baseline	89.855	89.540	1.004
512	baseline	199.971	200.245	0.999

TABLE VIII: OGBN-PRODUCTS: feature-width sweep.

F	choice	baseline (ms)	chosen (ms)	speedup
32	autosage	13.714	12.540	1.094
64	baseline	27.572	27.551	1.001
96	baseline	41.806	41.802	1.000
128	baseline	56.170	56.202	0.999
192	baseline	85.265	85.204	1.001
256	baseline	114.585	114.441	1.001
512	baseline	232.336	232.254	1.000

In both cases, AutoSAGE is most beneficial at small feature widths, and the benefit diminishes as the operation becomes bandwidth-bound.

E. Vec4 Vectorization and CTA-per-Hub Split

a) *Effect of vec4 vectorization*: Table VIII reports the effect of enabling `vec4` on cases where AutoSAGE has already chosen an optimized kernel.

TABLE IX: Vec4 ablation (speedup = OFF/ON; > 1 helps).

Dataset	F	Speedup
ER ($p=2 \times 10^{-5}$)	64	1.046
ER ($p=2 \times 10^{-5}$)	128	1.202
ER ($p=2 \times 10^{-5}$)	256	1.171
REDDIT	64	0.725

On the synthetic ER graph, vectorization consistently helps when the alignment and divisibility constraints are satisfied. On REDDIT at $F = 64$, vectorization hurts, and AutoSAGE correctly prefers non-vectorized kernels in that regime.

b) *CTA-per-hub split*: Table IX focuses on hub-skewed graphs at $F=128$ and compares the CTA-per-hub split to the baseline.

TABLE X: Split vs. baseline on hub-skewed graphs ($F = 128$).

Setting	Baseline (ms)	Split (ms)	Speedup
$N=20k$, hub= 5k, other= 64	1.603	0.717	2.236
$N=20k$, hub= 12k, other= 32	3.205	0.934	3.432
Sweep bests (two regimes)	—	—	2.985 / 4.563

These results confirm that explicitly handling hubs is important when a small fraction of rows accounts for a large portion of non-zeros.

F. Speedup Plots

To visualize the trends discussed above, we include several speedup plots for OGBN-PRODUCTS, REDDIT and the synthetic graphs.

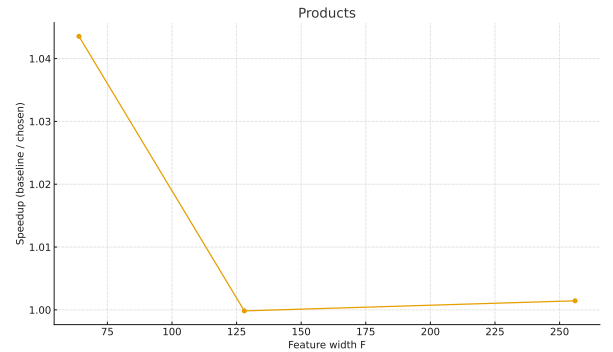


Fig. 1: OGBN-PRODUCTS: speedup vs. feature width F for selected settings.

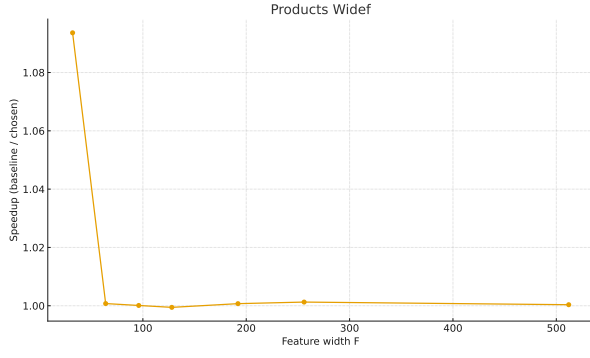


Fig. 2: OGBN-PRODUCTS: wider feature-width sweep.

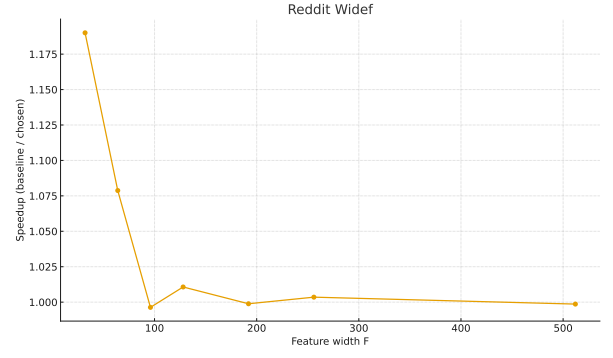


Fig. 5: REDDIT: wider feature-width sweep.

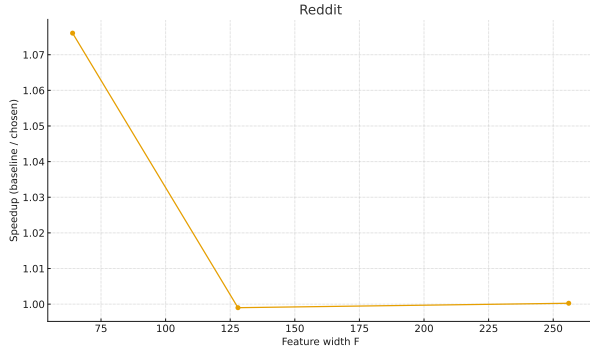


Fig. 3: REDDIT: speedup vs. feature width F with guardrail $\alpha = 0.95$.

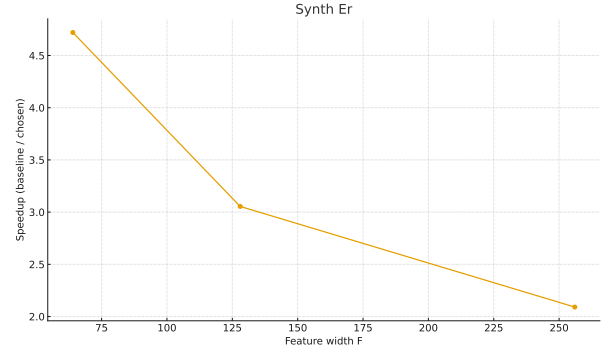


Fig. 6: Synthetic Erdős-Rényi graph: kernel-level speedups across feature widths.

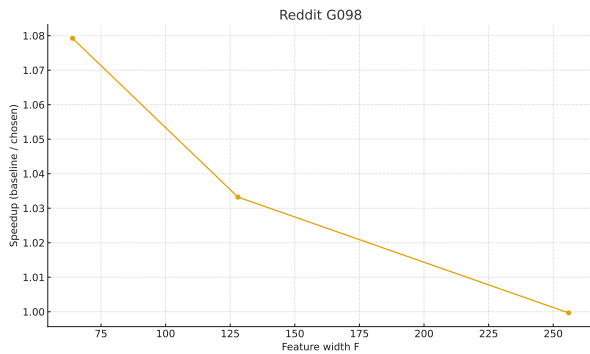


Fig. 4: REDDIT: speedup vs. feature width F with guardrail $\alpha = 0.98$.

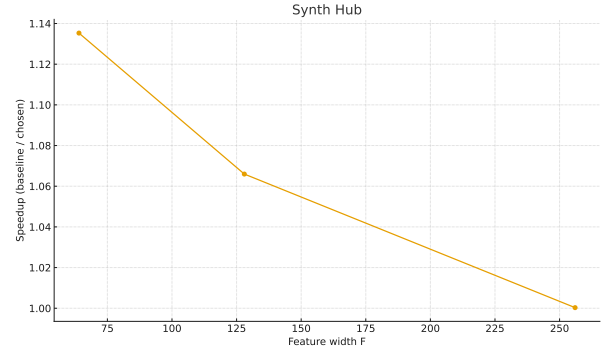


Fig. 7: Hub-skew synthetic graphs: effect of CTA-per-hub split on speedup.

G. Probe Overhead and CSR Attention

a) *Probe overhead*: With a probe fraction of 0.03 and a 1.000 ms time cap, the cost of probing is roughly 6–9% of a full-graph iteration on REDDIT at $F = 64$. A lower-overhead configuration (0.02, 0.500 ms) reduces this to about 3–4% at the expense of slightly noisier decisions. Once the cache is warmed up, replay mode avoids probing and the overhead becomes negligible.

b) *CSR attention pipeline*: We integrate `sddmm_csr_auto`, CSR row-softmax and

`spmm_csr_auto` into a full `csr_attention_forward` pipeline. In uncached mode, probe costs dominate the first few iterations, but after warm-up the pipeline runs at approximately 200.000ms on OGBN-PRODUCTS, with both SDDMM and SpMM choosing AutoSAGE kernels. Per-operation logs indicate SDDMM speedups of roughly 1.5–1.8 $\times$ and SpMM speedups of 5.2–5.5 $\times$ over their respective baselines in the regimes where AutoSAGE is active.

VII. DISCUSSION

A. When Does AutoSAGE Help?

Our experiments suggest that AutoSAGE is most beneficial when the graph has non-trivial sparsity or degree skew and the feature width is not so large that the operation becomes purely bandwidth-bound. In these cases, different mappings and tilings produce genuine performance trade-offs, and runtime information can help identify a good choice.

B. Practical Considerations

From a deployment perspective, AutoSAGE acts at the kernel level and does not require changes to model definitions or training loops. The guardrail provides a simple performance guarantee relative to a baseline, and the probe and cache mechanisms allow users to trade off exploration cost against reuse.

C. Limitations

AutoSAGE focuses on a narrow class of operators (CSR SpMM/SDDMM and CSR attention) and does not alter model semantics. Its benefits are largest on synthetic benchmarks and small feature widths; end-to-end improvements on full training pipelines depend on the fraction of time spent in these kernels. Our SDDMM baseline is a gather–dot implementation; incorporating alternative vendor or library kernels could change absolute gaps. Minor changes in drivers or CUDA versions may shift low-level timings; to mitigate this, the cache schema encodes device and toolchain versions.

VIII. CONCLUSION

We presented AutoSAGE, an input-aware CUDA scheduler for sparse GNN aggregation that combines analytical estimates, micro-probing and a guardrail into a practical system for selecting among a small set of kernel variants at runtime. AutoSAGE targets CSR SpMM and SDDMM and naturally supports a CSR attention pipeline. Experiments on standard benchmarks and synthetic graphs show that AutoSAGE can provide substantial speedups in sparse and skewed regimes while matching strong vendor baselines when operations are bandwidth-bound.

Future work includes extending AutoSAGE to additional sparse operations, exploring mixed-precision and `vec8` vectorization, and integrating AutoSAGE into larger GNN systems that optimize sampling, communication and scheduling jointly.

REFERENCES

- [1] G. Huang, G. Dai, Y. Wang, H. Yang, “GE-SpMM: General-purpose Sparse Matrix-Matrix Multiplication on GPUs for Graph Neural Networks,” arXiv:2007.03179, 2020.
- [2] M. K. Rahman, M. H. Sujon, A. Azad, “FusedMM: A Unified SDDMM-SpMM Kernel for Graph Embedding and Graph Neural Networks,” arXiv:2011.06391, 2020.
- [3] Y. Hu, Y. Huang, S. Li, Z. Jia, Z. Zhang, G.-Y. Wei, D. Brooks, Z. Zhu, Z. Zhang, “FeatGraph: A Flexible and Efficient Backend for Graph Neural Networks,” in Proc. SC, 2020.
- [4] Y. Wang, Z. Li, Q. Chen, S. Jiang, J. Xue, “GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs,” in Proc. OSDI, 2021.
- [5] Y. Zhou, J. Xiang, Z. Guo, Y. Sun, B. Cui, “AdaptGear: Accelerating GNN Training via Adaptive Subgraph-Level Kernels on GPUs,” in Proc. SIGMOD, 2023.
- [6] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” ICLR Workshop, 2019.
- [7] M. Wang et al., “Deep Graph Library: A Graph-Centric, Highly-Performant Package for GNN,” arXiv:1909.01315, 2019.
- [8] W. Hu et al., “Open Graph Benchmark: Datasets for Machine Learning on Graphs,” in Proc. NeurIPS, 2020.
- [9] T. Chen et al., “Learning to Optimize Tensor Programs,” in Proc. NeurIPS, 2018.
- [10] L. Zheng et al., “Ansor: Generating High-Performance Tensor Programs for Deep Learning,” in Proc. OSDI, 2020.
- [11] P. Tillet et al., “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” arXiv:1901.01986, 2019.
- [12] Y. Zhang, M. Yang, R. Baghdadi, S. Kamil, J. Shun, S. Amarasinghe, “GraphIt: A High-Performance Graph DSL,” Proc. ACM Program. Lang. (OOPSLA), 2018.
- [13] A. Brahmakshatriya et al., “Compiling Graph Applications for GPUs with GraphIt,” in Proc. CGO, 2021.
- [14] T. Gale et al., “Sparse GPU Kernels for Deep Learning,” arXiv:2006.10901, 2020.
- [15] C. Yang, A. Buluç, J. D. Owens, “GraphBLAST: A High-Performance Linear Algebra-based Graph Framework on the GPU,” ACM Trans. Math. Softw., vol. 48, no. 1, 2022.
- [16] R. Fan, W. Wang, X. Chu, “DTC-SpMM: Bridging the Gap in Accelerating General SpMM with Tensor Cores,” in Proc. ASPLOS, 2024.
- [17] Y. Xia et al., “Voltrix-SpMM: Sparse Matrix-Matrix Multiplication on Tensor Cores,” in Proc. USENIX ATC, 2025.
- [18] Q. Wang et al., “HR-SpMM: Adaptive Row Partitioning and Hybrid Kernel Design for Sparse Matrix Multiplication,” in Proc. ICS, 2025.
- [19] A. Jangda, S. Polisetty, A. Guha, M. Serafini, “NextDoor: Accelerating Graph Sampling on GPUs,” in Proc. EuroSys, 2021.
- [20] T. Liu et al., “BGL: GPU-Efficient GNN Training by Optimizing Graph Data I/O and Preprocessing,” in Proc. NSDI, 2023.
- [21] C. Wan et al., “PipeGCN: Efficient Full-Graph Training of GCNs with Pipelined Feature Communication,” in Proc. ICLR, 2022.
- [22] Y. Wang et al., “TC-GNN: Bridging Sparse GNN Computation and Dense Tensor Cores on GPUs,” in Proc. USENIX ATC, 2023.
- [23] Z. Xie et al., “Graphiler: Optimizing GNNs with Message Passing Data Flow Graph,” in Proc. MLSys, 2022.
- [24] H. Yuan et al., “A Comprehensive Evaluation of GNN Training Systems,” Proc. VLDB Endow., 2024.
- [25] S. Bajaj et al., “A Performance Comparison of Full-Graph and Mini-Batch GNN Training Systems,” Proc. VLDB Endow., 2025.
- [26] J. You, R. Ying, J. Leskovec, “Design Space for Graph Neural Networks,” in Proc. NeurIPS, 2020.